Lab 4: Linear Regression using Gradient Descent (NumPy and PyTorch)

Objective

The objective of this lab is to implement Linear Regression from first principles and understand how gradient descent optimizes model parameters.

Students will first build a Linear Regression model manually using NumPy (computing gradients and updating weights by hand), and then reimplement the same model using PyTorch's automatic differentiation and built-in optimizers.

Problem Statement

Linear Regression models the relationship between an independent variable X and a dependent variable y using the equation:

$$y = wX + b$$

where w is the weight (slope) and b is the bias (intercept).

The goal of training is to learn the values of w and b that minimize the error between predicted and actual values, using Gradient Descent and the Mean Squared Error (MSE) loss function:

$$\text{Loss} = (1/n)\ \Sigma\ (y\_actual - y\_predicted)^2$$

The gradients of the loss with respect to w and b are computed and used to iteratively update the parameters:

$$w = w - lr \times \partial Loss/\partial w$$
$$b = b - lr \times \partial Loss/\partial b$$

This lab is divided into two parts:

- Part A: Manual implementation of Linear Regression using **NumPy** (gradients derived and coded by hand).
- Part B: Implementation of the same problem using PyTorch's **`nn.Module`**, autograd, and optimizer.

Part A: Linear Regression with NumPy

Step 1: Import Required Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Generate Synthetic Data

```
X = np.arange(10, 30, 0.13)
y = 2.222*X + 3.333
```

Step 3: Train-Test Split

```
split = int(len(X)*0.8)
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]
```

Step 4: Visualize the Data

```
def plot_data(X_train, y_train, X_test, y_test, predictions=None):
    plt.scatter(X_train, y_train, c='b', label='Training Data')
    plt.scatter(X_test, y_test, c='r', label='Testing Data')
    if predictions is not None:
        plt.plot(X_test, predictions, c='g', label='Predictions')
    plt.xlabel('X'); plt.ylabel('y')
    plt.title('X vs y'); plt.legend(); plt.show()
```

Step 5: Initialize Parameters

```
w = np.random.randn()
b = np.random.randn()
```

Step 6: Define the Loss Function

```
def loss_fn(y_actual, y_out):
    return np.sum(np.square(y_actual - y_out)) / len(y_actual)
```

Step 7: Visualize Predictions Before Training

```
y_out = X_test*w + b
plot_data(predictions=y_out)
```

Step 8: Training Loop (Manual Gradient Descent)

```
lr = 0.1
epochs = 100
train_loss_list, test_loss_list = [], []

for epoch in range(epochs):
    # Forward pass
    train_preds = X_train*w + b
    train_loss = loss_fn(y_train, train_preds)
    # Compute gradients
    gradientW = (-2/len(X_train)) * np.sum((y_train - train_preds)*X_train)
    gradientb = (-2/len(X_train)) * np.sum(y_train - train_preds)
    # Update parameters
    w = w - gradientW*lr
    b = b - gradientb*lr
    # Evaluate on test data
    test_preds = X_test*w + b
    test_loss = loss_fn(y_test, test_preds)
    train_loss_list.append(train_loss)
    test_loss_list.append(test_loss)
```

Step 9: Visualize Predictions After Training

```
y_out = X_test*w + b
plot_data(predictions=y_out)
```

Step 10: Plot Loss Curves

```
plt.plot(range(epochs), train_loss_list, label="Training Loss")
plt.plot(range(epochs), test_loss_list, label="Testing Loss")
plt.title("Epochs vs Loss");
plt.xlabel("Epochs");
plt.ylabel("Loss")
plt.legend();
plt.show()
```

Step 11: Check the Learned Parameters

```
print(f'Finally w={w} and b={b}')
```

Part B: Linear Regression with PyTorch

Step 1: Import Libraries and Check Device

```
import torch
from torch import nn


if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
```

Step 2: Generate Data and Convert to Tensors

```
X = np.arange(0, 1, 0.02)
y = 5.66*X + 3.227


X_train, y_train = torch.from_numpy(X_train), torch.from_numpy(y_train)
X_test, y_test = torch.from_numpy(X_test), torch.from_numpy(y_test)
```

Step 3: Define the Model Class

```
class LinearRegressionNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.weights = nn.Parameter(torch.randn(1, requires_grad=True,
           dtype=torch.float))
        self.bias = nn.Parameter(torch.randn(1, requires_grad=True,
           dtype=torch.float))
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.weights*x + self.bias
```

Step 4: Instantiate the Model

```
torch.manual_seed(42)
model_1 = LinearRegressionNetwork()
model_1.state_dict()
```

Step 5: Predictions Before Training

```
with torch.inference_mode():
    y_pred = model_1(X_test)
plot_data(predictions=y_pred)
```

Step 6: Define Loss Function and Optimizer

```
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(params=model_1.parameters(), lr=0.1)
```

Step 7: Training Loop

```
epochs = 200
train_loss_list, test_loss_list = [], []

for epoch in range(epochs):
    model_1.train()
    train_pred = model_1(X_train)
    loss = loss_fn(y_train, train_pred)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

```
    model_1.eval()
    with torch.inference_mode():
        test_pred = model_1(X_test)
        test_loss = loss_fn(y_test, test_pred)

    train_loss_list.append(loss.detach().numpy())
    test_loss_list.append(test_loss.detach().numpy())
```

Step 8: Predictions After Training

```
with torch.inference_mode():
    y_pred = model_1(X_test)
plot_data(predictions=y_pred)
```

Step 9: Plot Loss Curves

```
plt.plot(range(epochs), train_loss_list, label="Training Loss")
plt.plot(range(epochs), test_loss_list, label="Testing Loss")
plt.title("Epochs vs Loss");
plt.xlabel("Epochs");
plt.ylabel("Loss")
plt.legend();
plt.show()
```

Tasks

1. Implement the manual NumPy version of Linear Regression and record the learned values of w and b. Compare them with the true values used to generate the data.
2. Implement the PyTorch version using **`nn.Module`** and compare its learned parameters with Part A.
3. Plot and compare the loss curves (training vs testing) for both implementations. Comment on convergence speed.
4. Experiment with different learning rates (e.g., 0.001, 0.01, 0.1, 0.5) and observe their effect on convergence and stability.
5. Experiment with different numbers of epochs and determine the minimum epochs needed for convergence in each implementation.
6. Modify the train-test split ratio (70-30, 80-20, 90-10) and compare the final test loss for each split.
7. Replace the optimizer in Part B with **`torch.optim.Adam`** and compare its convergence behavior with SGD.
8. Add Gaussian noise to the synthetic data **(`y = wX + b + noise`)** and analyze how it affects the learned parameters and loss curves.
9. Extend the model to Multiple Linear Regression using two or more input features and evaluate performance.
10. Save the trained PyTorch model, reload it, and verify that predictions match.
11. Discuss the advantages of using PyTorch's autograd and optimizers over manually computing gradients.